

OCP (Open/Closed Principle)

Суть принципа: Классы должны быть открыты для расширения, но закрыты для модификации.

Обоснование соблюдения в проекте:

В проекте используются абстрактные классы (интерфейсы), что позволяет добавлять новые реализации без изменения существующего кода:

```
# Существующий код (закрыт для модификации)
class ComicTranslator:
    def __init__(self, ocr: OCRProvider): # Зависимость от абстракции
        self.ocr = ocr

# Можно добавить новую реализацию (открыт для расширения)
class TesseractOCRProvider(OCRProvider): # Новая реализация
    def recognize(self, image_path):
        # Реализация через Tesseract
        pass

# Код ComicTranslator не требует изменений!
```

Пример нарушения

```
class BadTranslator:
    def translate(self, text, service_type):
        if service_type == 'google':
            # Google API
            pass
        elif service_type == 'yandex': # Новый сервис = новая ветка
            # Yandex API
            pass
        elif service_type == 'deepl': # Каждый раз добавляем ветку
            # DeepL API
            pass
        # ✗ При добавлении сервиса нужно менять код класса
```

LSP (Liskov Substitution Principle)

Суть принципа: Объекты дочернего класса должны быть заменяемы объектами родительского класса.

Обоснование соблюдения в проекте:

Любая реализация интерфейса OCRProvider может быть подставлена без изменения логики:

Функция работает с любой реализацией OCR

```
def process_comic(ocr: OCRProvider):
    regions = ocr.recognize('comic.png') # Не важно, какая реализация
    return regions

# Обе реализации взаимозаменяемы
process_comic(EasyOCRProvider()) # ☒ Работает
process_comic(TesseractOCRProvider()) # ☒ Работает с той же логикой
```

Пример нарушения

```
class BadOCRProvider(OCRProvider):
    def recognize(self, image_path):
        if not os.path.exists(image_path):
            raise FileNotFoundError("Новый тип исключения!")
        # ✗ Родительский класс не выбрасывал это исключение
        # ✗ Клиентский код может не обработать новое исключение
```

ISP (Interface Segregation Principle)

Суть принципа: Интерфейсы должны быть узкоспециализированными.

Обоснование соблюдения в проекте:

В проекте созданы узкие интерфейсы:

```
# Только для OCR
class OCRProvider(ABC):
    @abstractmethod
    def recognize(self, image_path): pass

# Только для перевода
class TranslatorProvider(ABC):
    @abstractmethod
    def translate(self, text): pass

# Только для обработки изображений
class ImageProcessor(ABC):
    @abstractmethod
    def draw_text(self, image, regions): pass
```

Пример нарушения

```
class IAllInOne(ABC):
    @abstractmethod
    def recognize(self): pass
    @abstractmethod
    def translate(self): pass
    @abstractmethod
```

```
def draw_text(self): pass
# ✗ OCR провайдер вынужден реализовывать translate() и draw_text()
# ✗ Переводчик вынужден реализовывать recognize() и draw_text()
```

DIP (Dependency Inversion Principle)

Суть принципа: Модули верхних уровней не должны зависеть от модулей нижних уровней.

Обоснование соблюдения в проекте:

```
class ComicTranslator:
    def __init__(self, ocr: OCRProvider, translator: TranslatorProvider):
        # ☑ Зависимости от абстракций, а не от конкретных классов
        self.ocr = ocr
        self.translator = translator

# Внедрение зависимостей (Dependency Injection)
ocr = EasyOCRProvider()          # Конкретная реализация
translator = GoogleTranslateProvider()
app = ComicTranslator(ocr, translator) # Зависимости передаются извне
```

Пример нарушения

```
class BadComicTranslator:
    def __init__(self):
        # ✗ Жёсткая привязка к конкретным классам
        self.ocr = EasyOCRProvider()
        self.translator = GoogleTranslateProvider()
        # ✗ Нельзя заменить на другие реализации без изменения кода
```

Принцип KISS (Keep It Simple, Stupid)

Суть принципа: Программа должна быть максимально простой и понятной.

Обоснование соблюдения в проекте:

Аспект Реализация в проекте

Имена переменных: input_path, output_path, text, regions — понятны без комментариев

Размер методов: Каждый метод выполняет одно действие и занимает не более 20-30 строк

Уровень абстракции: Один уровень абстракции на метод

Управление потоком: Простые условия if/else, без излишней вложенности

```
def _wrap_text(self, text: str, max_width: int) -> List[str]:
```

```

"""Переносит текст на новые строки — 15 строк простого кода"""
words = text.split()
lines = []
current = []
for word in words:
    current.append(word)
    if len(' '.join(current)) * 10 > max_width - 20:
        current.pop()
        if current:
            lines.append(' '.join(current))
            current = [word]
if current:
    lines.append(' '.join(current))
return lines

```

Пример нарушения

```

def wrap_text_very_complex(self, text, width, font, draw, consider_hyphens,
                             consider_punctuation, max_lines, min_font_size,
                             adjust_by_letter, use_ligatures, debug_mode):
    # 100+ строк сложной логики
    # ✗ Слишком много параметров
    # ✗ Слишком сложно для простой задачи
    pass

```

Принцип DRY (Don't Repeat Yourself)

Суть принципа: Избегайте дублирования кода.

Обоснование соблюдения в проекте:

Создание папок - FileManager.ensure_directories()

Сканирование файлов- FileManager.scan_images()

Группировка областей - group_regions() и _merge_regions()

Перенос текста - wrap_text()

Ленивая загрузка моделей - get_reader(), _get_font()

Пример устранения дублирования:

```

# Единый метод для перевода любого списка файлов
def translate_all(self, files: List[str]):
    for file in files:
        input_path = os.path.join('to_translate', file)
        output_path = os.path.join('translated', f"translated_{file}")
        self.translator.translate(input_path, output_path)

# Вызывается из разных мест, но код один

```

```
FileManager.translate_all(translator, all_files) # Все файлы
```

Пример нарушения

```
def translate_all_files(self):
    files = os.listdir('to_translate')
    for f in files:
        if f.endswith('.png'):
            img = cv2.imread(f'to_translate/{f}')
            text = pytesseract.image_to_string(img)
            # ... 50 строк кода

def translate_selected_files(self, indices):
    files = os.listdir('to_translate')
    for i in indices:
        f = files[i]
        if f.endswith('.png'):
            img = cv2.imread(f'to_translate/{f}')
            text = pytesseract.image_to_string(img) # X Тот же код!
            # ... те же 50 строк кода
```

SRP (Single Responsibility Principle)

Принцип единственной ответственности

Формулировка: "У каждого класса должна быть только одна причина

для изменения"

Класс	За что отвечает	Почему одна причина для изменения
EasyOCRProvider	Распознавание текста на изображении	Изменение OCR движка не затрагивает перевод и отрисовку
GoogleTranslateProvider	Перевод текста с английского на русский	Смена API перевода не влияет на распознавание
PILImageProcessor	Отрисовка текста на изображении	Изменение шрифтов не ломает OCR

ComicTranslator	Координация работы всех модулей	Оркестрация не смешивается с бизнес-логикой
FileManager	Операции с файлами и папками	Работа с диском отделена от перевода
ConsoleMenu	Пользовательское меню и ввод	UI не зависит от логики перевода

Пример кода, соблюдающего SRP

```
class EasyOCRProvider(OCRProvider):
    """Только распознавание текста - одна причина для изменения"""

    def recognize(self, image_path: str) -> List[TextRegion]:
        reader = self._get_reader()
        img = cv2.imread(image_path)
        results = reader.readtext(img, detail=1)
        # ... только логика распознавания
        return regions

class GoogleTranslateProvider(TranslatorProvider):
    """Только перевод текста - одна причина для изменения"""

    def translate(self, text: str) -> str:
        return self.translator.translate(text)

class PILImageProcessor(ImageProcessor):
    """Только отрисовка - одна причина для изменения"""

    def draw_text(self, image, regions):
        # ... только логика рисования
        return result
```

Пример нарушения

```
class BadComicTranslator:
    """Нарушение SRP - класс делает всё сразу!"""

    def translate_comic(self, image_path):
        # 1. Распознавание текста
        img = cv2.imread(image_path)
```

```
reader = easyocr.Reader(['en'])
results = reader.readtext(img)

# 2. Перевод текста
translator = Translator()
translated = translator.translate(text)

# 3. Отрисовка
result = Image.new('RGB', (800, 600), 'white')
draw = ImageDraw.Draw(result)
draw.text((50, 50), translated)
result.save('output.png')

# 4. Файловые операции
import shutil
shutil.copy('output.png', 'backup.png')

# ✗ ЧЕТЫРЕ причины для изменения в одном классе!
```